

ELEC 378 Spring 2025

Final Project Report

May 6th, 2025

Ankit Burudgunte, Anthony Zhang, & John David Villarreal

1. Data Exploration

We started by listening to some files in the training set. We noticed that the same sentences weren't used for each recording, and that within a single class, different sentences were spoken. There was some silence (or at least, periods without speech) at the beginning and end, and of course between words. The recordings also seemed clean (relatively low noise as far as we could hear). Finally, we thought the differences in emotion seemed relatively subtle sometimes.

We then graphed the raw audio (amplitude vs. time), FFT of this audio (in the frequency domain), and PSD vs. frequency of the first .wav file in the training set from each emotion category. This showed some basic trends in the audio. We could visually represent the silence in the files, which seemed to take up a significant amount of time for each audio file we graphed. Also, in the PSD graph, which generally shows the 'strength' of the signal at each frequency, the graph showed that emotions one would expect to have a stronger signal for, like 'angry', did in fact have higher PSDs for many frequencies than for an emotion like 'sad', which you might expect to be softer.

2. Feature Extraction

Planning

We started our investigation into the best features by running the TA-provided SVM model with minimal changes on the features provided at the time. These features were MFCCs and Chroma as calculated by the librosa feature extraction methods, with the audio timeseries, sampling rate, and for the MFCCs, the number of coefficients, which was set to a standard value for speech tasks of 20. On our check of average accuracy through cross-validation, this base model performed at around 60%, which matched the accuracy reached on the public leaderboard on Kaggle. Based on this, we knew we could add features.

After reading a paper on emotion recognition¹, we found that it is important to include spectral, prosodic, and speech quality features to fully capture emotional content. Spectral features aim to capture characteristics of the data in the frequency domain, and often show how energy is distributed across frequencies. Prosodic features are melodic and rhythmic features of speech, like stresses, speech timing, and pitch. Speech quality features include general characteristics of sound and may capture features related to the shape of the vocal organs. These categories will be discussed in more detail in the final list of features we selected.

We began adding features from each category based on their librosa descriptions and their use in the paper. We used a trial-and-error process, starting with a variety of features we expected to be effective and plugging them into the models to gauge which were actually effective. In the end, we came up with a final list of features, documented below. We have also included a discussion of which features we chose to remove later in the overfitting section.

MFCC Features²

A. MFCCs

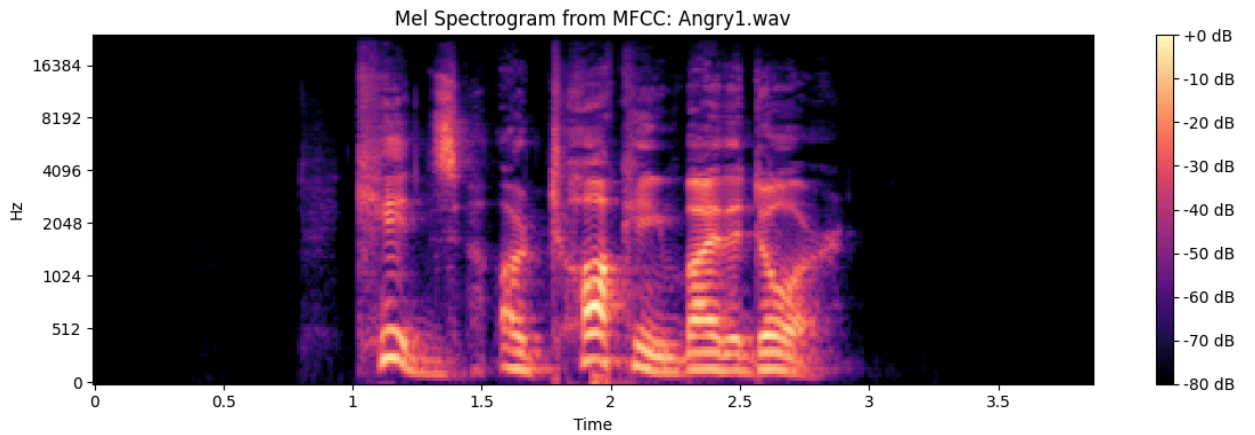
Mel-Frequency Cepstral Coefficients (MFCCs) capture the short-term power spectrum of a sound. They are calculated by the following steps:

1. STFT: Find the Short-Time Fourier Transform of a windowed excerpt of a signal to produce a spectrogram in the standard frequency.
2. Convert the power spectrum to the mel scale from the standard frequency. One formula for converting from f hertz to m mels is

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

3. Take the logs of the powers at each mel frequency.
4. Take the discrete cosine transform (DCT) of the list of mel log powers.

This is a plot of a Mel Spectrogram used for MFCC extraction on the 1st file in the Angry training set:



The amplitudes of the resulting spectrum are the MFCCs. These are a useful measure because the mel scale mimics human hearing, which processes sound on a logarithmic scale. Mimicking human hearing is important for emotion detection because emotions influence how we perceive voices.

B. Deltas⁴

The first difference of the MFCCs at two adjacent times:

$$\Delta_t = f_t - f_{t-1}$$

C. Delta-Deltas⁴

$$\Delta\Delta_t = \Delta_t - \Delta_{t-1}$$

Delta-Deltas, along with the Deltas, capture changes over time in the MFCCs, which are also important to the identification of emotion. Deltas and Delta-Deltas approximate the 1st and 2nd derivatives of the MFCCs, respectively. These become especially important when we

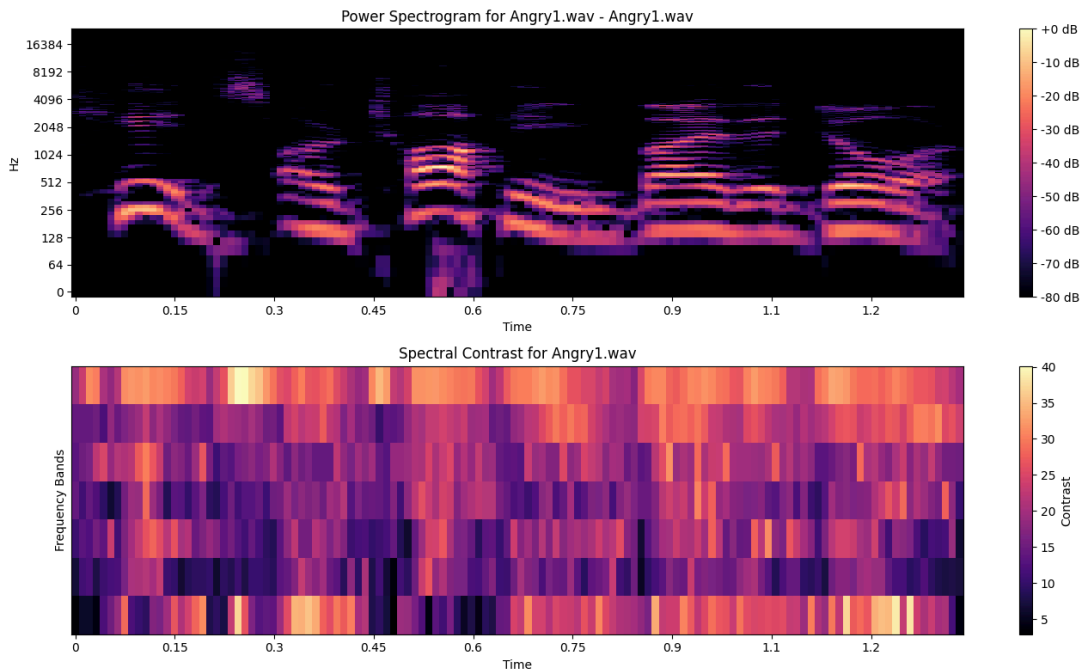
aggregate features by mean, or other single-number measures, and become unable to check the changes in MFCCs over time throughout a particular signal.

Spectral Features¹

Generally, spectral features capture characteristics of the data in the frequency domain. The way energy is distributed across frequencies is generally important to human perception of the sounds.

A. Spectral Contrast

The difference in amplitude between peaks and valleys in different frequency sub-bands. This distinguishes harmonic vs. noisy sounds. Below are the plots of the power spectrogram and spectral contrast for the 1st file in the Angry training set:



B. Spectral Centroid

The weighted average frequency of the spectrum, which represents the ‘center of mass.’

It is calculated by $\text{Centroid} = \frac{\sum_k f_k S_k}{\sum_k S_k}$ where f_k = frequency (as a discrete bin) and S_k = magnitude at bin k .

C. Spectral Bandwidth

Measures the frequency range where most of the signal’s energy is concentrated, which reflects the spread of energy. The wider the bandwidth, the more spread-out energy is across all the frequencies.

D. Spectral Flatness

Quantifies how ‘noise-like’ vs. ‘tonal’ (speechlike) a sound is (closer to 1 = noise, while closer to 0 = tonal). This identifies noisy (unclear) segments like whispering or yelling, which vary across emotions.

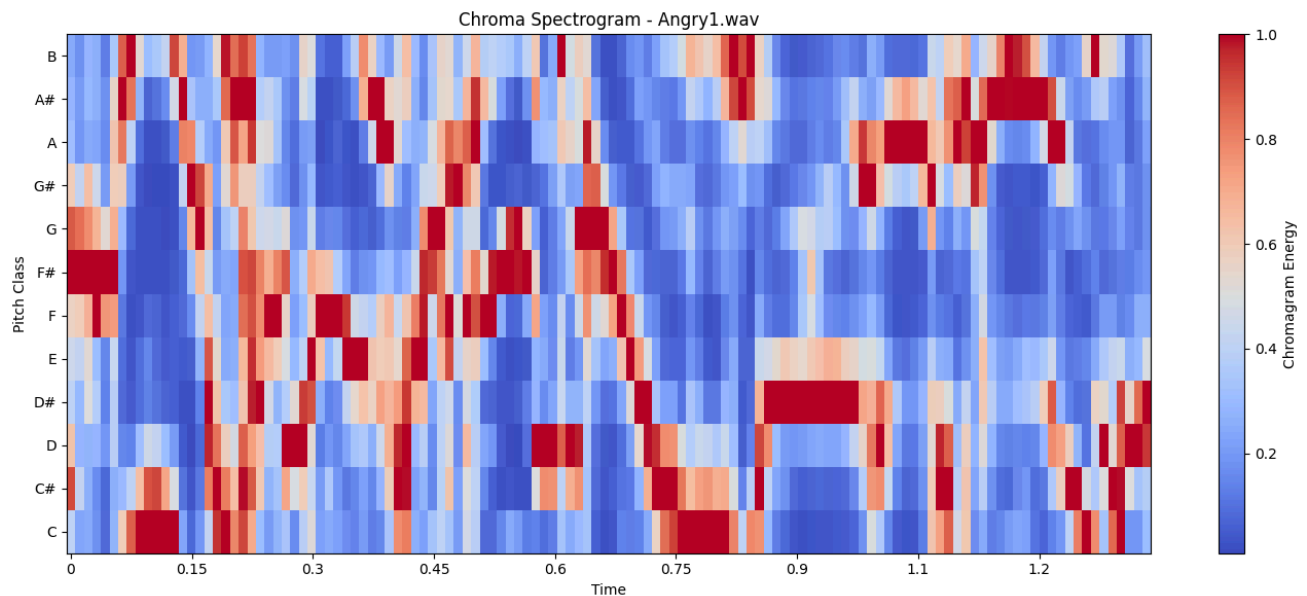
E. Spectral Rolloff

The frequency below which a specified percentage of the total energy (in our case, 85% by default) resides.

Other Features

A. Chroma

The 12 musical notes of Western music, like (C, C#, D, ...) This maps frequencies to 12 classes defined based on pitch, with no distinctions between different octaves (meaning C4 and C5 are both mapped to the C class). It is useful because how harmonic and/or tonal a voice is can be affected by emotion, such as sounding ‘sing-songy’ when happy. Here is a spectrogram of the Chroma for the 1st training data file in the Angry folder:



B. RMS

The root mean squared (RMS) is the average magnitude of the signal. It gives a measure of the average power (loudness/intensity) over time. Changes in the RMS over time indicate stresses and/or emphasis.

C. Zero-Crossing Rate

The Zero-Crossing Rate (ZCR) is the rate at which the signal crosses 0, or changes sign. This distinguishes voiced vs. unvoiced sounds, which are useful for finding quick changes or

finding linguistic patterns related to the way words are pronounced that might be affected by emotion.

Not Included Features

We did not explicitly include prosodic features, such as pitch and magnitude. We were already including a relatively large number of features, and with more, it took a very long time to run the code and generate predictions.

Also, after trying different combinations, it seemed like prosodic features were not increasing accuracy by much. This may have been due to the lower effectiveness of prosodic features for emotion classification specifically¹, as opposed to other audio classification tasks. This is stated in section 4.3 of the paper on emotion recognition using CNNs. Generally, emotion recognition requires detailed frequency domain features due to the large amount of emotional information which is related to features that vary with frequency, while prosodic features are often in the time domain. Finally, specifically for the CNN, spectrograms represent the frequency domain, so it makes sense to produce the necessary spectrograms from frequency domain features.

Preprocessing

We used feature extraction from the librosa library for this project. After extracting the features we wanted, we reduced dimensionality by aggregating features by mean, sometimes standard deviation, and a few other measures, and stacked the measures into a feature vector. This also had the benefit of converting audio files of various lengths (meaning we had extracted raw features of various lengths) to fixed-length vectors.

Later in our process, we decided to include a bandpass Butterworth filter to remove frequencies below 20 Hz and above 20,000 Hz. We believed these two steps would eliminate some of the noise in the audio files (even if it was generally low) and better help the model identify the emotions with clearer data, which might lead to less fitting to noise.

While implementing each model, we first normalized and scaled our data using the `StandardScaler()` function provided by `sci-kit learn`. While implementing the CNN specifically, we trimmed audio files for silence before generating their mel spectrograms, which was a strategy to significantly reduce processing time as there were large periods of silence in the audio files.

3. Model Selection

3.1 Support Vector Machine

Support Vector Machines excel in creating decision boundaries to classify linearly separable data. Using kernels to complement SVMs allow us to map data vectors into higher dimensional features spaces and accurately classify even when the data is not linearly separable. Kernel SVMs optimize a hyperplane by maximizing the margin between two support vectors

while minimizing hinge loss through penalizing misclassifications and margin violations. The optimization function is given below.

$$\min_{w,b} = \left[\frac{1}{2} ||w||^2 + C \sum_{i=1}^n \xi_i \right]$$

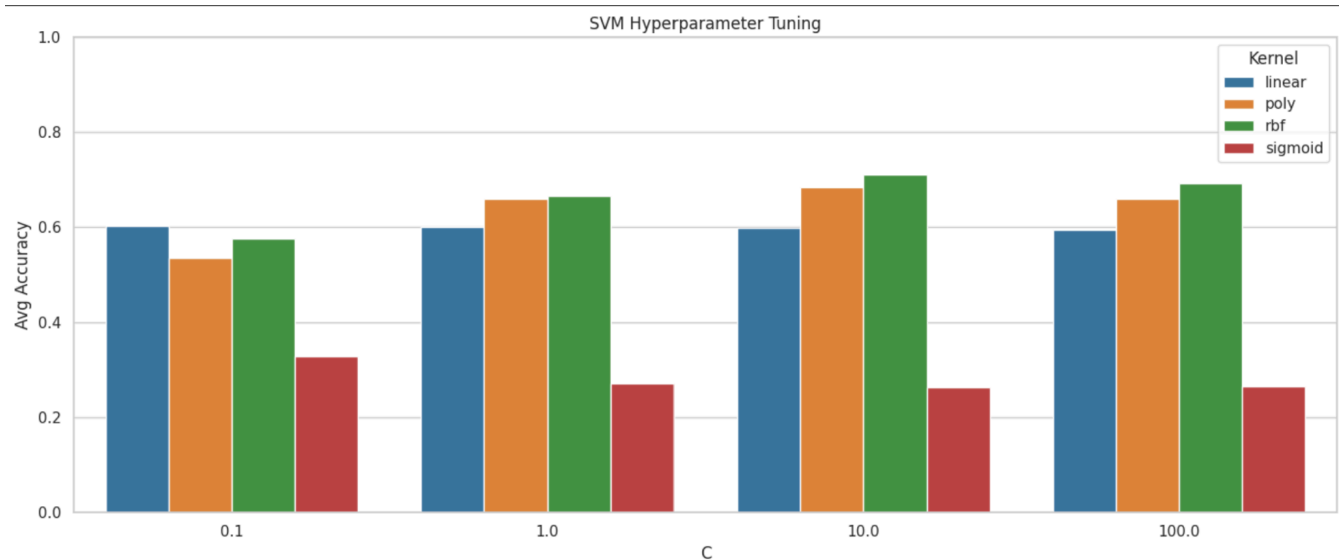
Hyperparameters

In order to optimize the SVM model, various variables such as the kernel choice, regularization parameter and gamma weight can be adjusted. Beginning with the kernel choice, our different choices included the linear kernel, polynomial kernel, radial basis function kernel, and sigmoid kernel. Since both the radial basis function kernel and sigmoid kernel mapped the data into an infinite number of dimensions, we believed these two choices to result in the optimal SVM model, as most forms of data become linearly separable in higher dimensions. The regularization parameter, denoted as C, controls the tradeoff between the hinge loss and margin size. As C increases, regularization and margin size decreases, leading to possible overfitting of the training data. Alternatively, as C decreases, the hinge loss decreases, which helps to create a smoother decision boundary at the expense of possibly underfitting the data. The gamma parameter, denoted as γ , determines the influence of each single training point in shaping the decision boundary. A small gamma allows for a smoother decision boundary with the risk of underfitting while a large gamma risks overfitting but adapts more to each data point.

To determine the optimal hyperparameters, we used GridSearch CV to test each possible combination of parameters over the 5-fold cross-validation train-test data and determine which results in the highest accuracy.

The tested parameters for each hyperparameter was as follows:

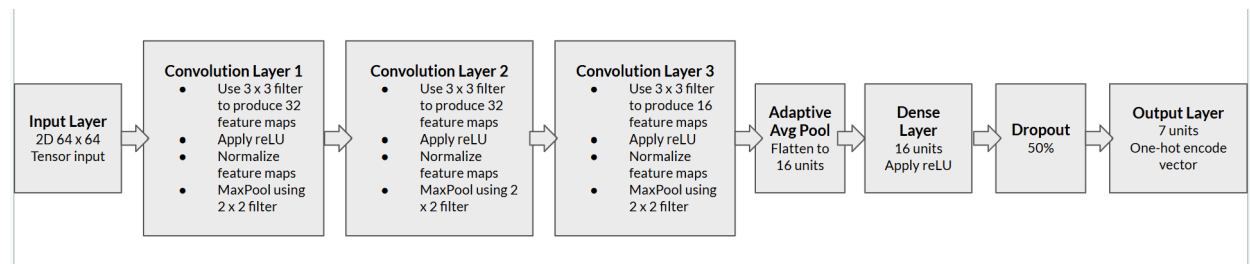
- Kernels: rbf, linear, polynomial, and sigmoid
- C: 0.1, 1, 10, and 100
- Gamma: scale or auto



Overall, through testing, we determined that the optimal kernel for SVM was rbf, as it consistently performed better on training data. When the rbf kernel was used, accuracy was maximized when $C = 10$, and as such, we chose to use those hyperparameters. Finally, scale performed marginally better than auto, which led us to choosing that, even though the difference was almost negligible. When testing on the Kaggle dataset, our results suggested that we had chosen correctly, since the accuracy of 72.535% matched very closely to our train accuracy of around 70%.

3.2 Convolutional Neural Network

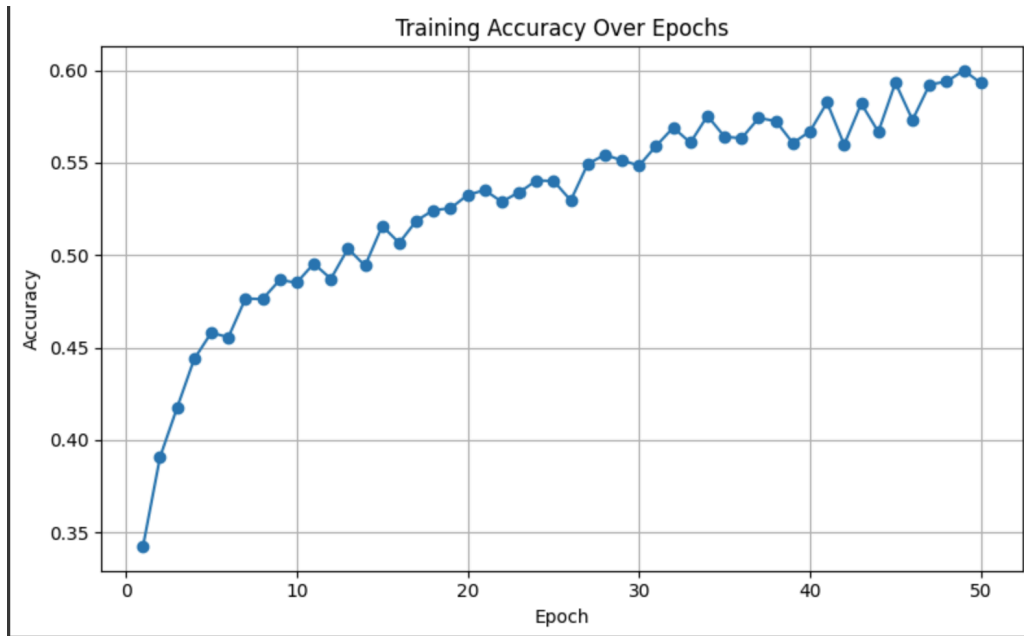
Convolutional Neural Networks (CNNs) are a type of neural network that specializes in image or 2d array analysis. This is due to the fact that CNNs apply filters or convolutions on the input data to detect local patterns such as shapes or edges in the image, thereby enabling it to learn from the data effectively. In the case of this project, since audio files can be represented visually using mel-spectrograms, CNNs can perform well when learning frequency or time patterns in the data. We hoped that CNNs would be able to more accurately distinguish between different emotions due to its large number of parameters that typically come with a deep neural network.



We started out our CNN model by arbitrarily selecting convolution layer sizes and iteratively determining what to add or change. Through this method, we ended up with a relatively strong model that matched the accuracy of our SVM model. The architecture is shown visually in the image above. The CNN begins by taking in a 2D Tensor of size 64 by 64. It applies a convolution layer using a 3 x 3 filter to produce 32 feature maps, before applying reLU, normalizing, and max pooling to prevent overfitting. The second and third convolution layer apply the same functions to the data, just resulting in a different sized feature map. This choice was to expand the number of features to be used in the model, which we believed would help learn patterns in the data that may hold the key to achieving a higher accuracy. Finally, we would flatten these feature maps, before putting the data through a multilayer perceptron network. Right before the output layer, we would zero out 50% of the weights to prevent overfitting. The output layer would return a 7 dimensional vector, which we could then one-hot encode to determine which emotion was predicted by the model. In total, our model contained 14,743 parameters.

During epoch training, we noticed that around 40-50 epochs, the model would be very close to convergence, only increasing by a slight margin after this range. As each epoch took 1-2 minutes, we decided to cut off the epoch training to 50 in order to save time. We decided to train each epoch using the Adam optimizer, which is a widely accepted optimizer used in neural

networks. We chose Adam as the adaptive learning rate and momentum would allow for a more stable convergence, meaning we were more likely to approach a local minimum on our loss function to best optimize our CNN model. Additionally, our CNN mode had an extremely non-convex gradient (from previous unsuccessful attempts to get the CNN to converge), and Adam has been shown to work well in optimizing noisy gradients.



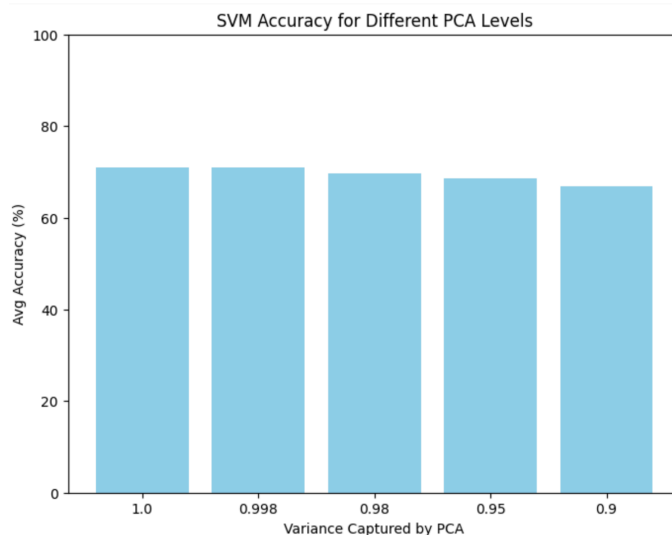
The results of our final trained CNN model is shown above. After 50 epochs, the accuracy on the train data is around 60%. Unfortunately, due to the long convergence time of CNN models ranging from 1 to 3 hours, optimizing parameters to further improve our CNN model was limited, and we could only test very small changes to the model, from differently sized convolutional layers to varying the learning rate of the Adam optimizer. As such, while we were able to sometimes reach more than 80% accuracy on the training data, the CNN model never performed better than 62.47% on the Kaggle dataset. This shows that our CNN model is overfitting the data in some steps, and we believe that with more research on popular CNN models, we could identify the issue. One potential solution that we believe could solve the overfitting issue is including a dropoff parameter after each convolutional layer to ensure the CNN is not relying on certain weights and overfitting to them. However, due to the cost of training and in the best interest of our Kaggle competition performance, we decided that attempting to fix this issue was not a good use of time and prioritized focusing on optimizing our SVM model.

4. Overfitting Considerations

In order to mitigate the risk of overfitting to the test data, we used K-fold cross validation to split our test data into 5 different train-test data sets. K-fold cross validation works by partitioning the data into 5 evenly distributed groups. For each partitioned group of data, we would train our classifier on the rest of the data and then test for its accuracy on groups not trained on. By computing the average performance of our classifier over all these folds, we could

determine whether or not the performance of the model was due to overfitting. A consistent accuracy for all folds suggests that the classifier would not overfit on the data.

An additional step taken to reduce overfitting involved Principal Component Analysis (PCA) on the data before training the classifier in order to reduce the total variation of the data. PCA finds the principal components in the data that capture the most variance. By selecting the principal components that capture a majority of the variation and removing the rest, we can remove unnecessary noise in the data that would otherwise cause our model to overfit. PCA additionally provides a benefit in that by reducing unnecessary principal components, it is effectively reducing the dimensionality of the data, which reduces model complexity and can decrease the computational complexity of training the classifier on the data.



Based on testing various PCA parameters on our SVM model, we determined that the overall accuracy on the training data would increase as the variance captured by PCA increases. However, this comes at a tradeoff in potentially overfitting the data. Therefore, we decided to test the trained models on Kaggle test data to determine the dropoff in overfitting. When submitting on Kaggle, we discovered that setting the PCA hyperparameter to 0.998 would allow for a balance between having higher accuracy and being generalizable to new data.

In the SVM model, the dimensionality of features after feature extraction was 183. We believed this to be too much and would cause the SVM model to overfit, and as such, utilized recursive feature extraction to remove features until the dimensionality of the features was reduced to 100. For the recursive feature extraction, we used the Random Forest Classifier⁵ to determine the importance of features. While we believe 100 dimensions to still be too much, our results suggested that reducing the feature dimensionality even further would reduce model accuracy, and thus we kept it as such. Given that the dataset has around 13,000 audio files, extracting 100 numbers for each as data points is rather reasonable, and can work well with a kernel SVM that is able to further increase the dimensionality of the data to make it near-linearly separable.

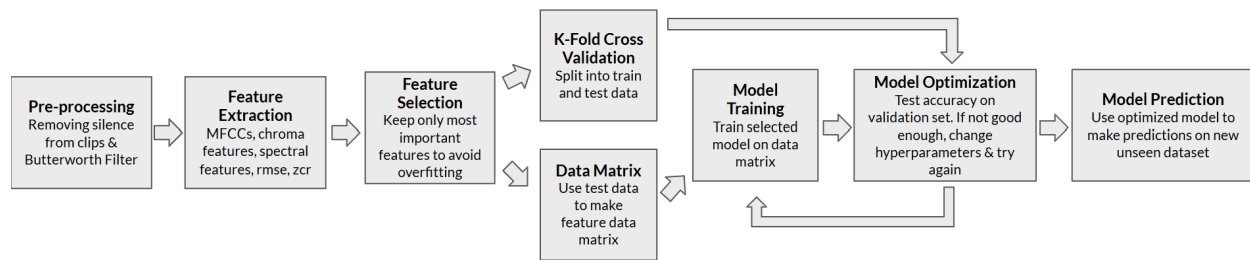
In the case of the CNN model, the only feature the model is trained on is the mel-spectrogram. Through preprocessing techniques such as deleting silent audio parts of the file and average-pooling the mel-spectrogram matrix, we can reduce the dimensionality of the data into a 64 by 64 2D array. As discussed previously, our CNN model contains 14,743 parameters. Since our dataset is only around 13,000 audio files, we believe that the number of parameters in the CNN model may be excessive; however, rather long stochastic gradient descent optimization times prevented us from efficiently optimizing the selection of parameters. Due to the large number of parameters, we believe that the CNN model would easily overfit to the training data, which proved to be true through initial testing of various architectures. As such, we decided to implement a number of strategies to reduce overfitting.

Between each convolutional layer of the CNN model, we used BatchNorm2d to apply a regularization to the weights and stabilize the SGD loss function. Similarly, between convolutional layers, we also used MaxPool2d to reduce the dimensionality of the model, thus reducing the number of parameters to be used in future layers and help to prevent overfitting. Towards the end of the model, when the neural network is simplifying just a multilayer perceptron network, we also chose to include a dropout function to randomly alter half of the weights of the network to 0, which further reduces possible overfitting from the training of the data. Finally, in order to reduce overfitting, we would split the training data into mini-batches so that in between epochs, we would use a different mini-batches to train the model on. This would effectively prevent the model from overtraining on a specific subset of the data for too long and improve generalization.

Overall, we found that preventing overfitting for SVM worked very well, but not for CNN models. Initially, for CNNs with lower accuracies in the range of 60%, the model would generalize well to unseen data. However, with longer training time, model accuracy on training data would increase, while accuracy on test data would remain the same. Specifically, in one instance, our CNN model had an 85% accuracy on train data, but when submitted to Kaggle, the accuracy on the test data was 62%. This meant that while the model fit to the training data too much and did not generalize well. Since the CNN model took a long time to train, we were unable to fully solve this issue and instead overall pivoted to focusing on improving the accuracy of the SVM model.

5. Complete Pipeline

The overall pipeline of our project is shown in the diagram below.



Pre-processing

In this step, we decided to remove silence from the audio files in order to shorten the audio time series and reduce the computational complexity required to train our models. Late in the project timeline, we also decided to include a bandpass Butterworth filter to remove frequencies below 20 Hz and above 20,000 Hz. We believed these two steps would eliminate some of the noise in the audio files and better help with the generalization of the model to data it hasn't seen before.

Feature Extraction

In this step, we selected features that seemed to be important based on prior research done in the field of audio data processing. Features such as MFCCs, chroma features, rmse, spectral features were added iteratively and then tested to determine if the classification model would improve with the new addition. Some features such as pitch were also added in this step but removed due to the large computational time it took to process that feature for each data.

Feature Selection

After selecting features, we saw that our feature vector dimensionality was 183. We believed this to be too high and would cause overfitting, so we decided to use recursive feature extraction to reduce the dimensions of our features to 100. By letting the model choose which features were important based on its ability to capture variance in the data, we are able to prevent any necessary data from being accidentally removed. We also tested removing more or less vector dimensions to determine the optimal number of features to remove (which we decided was 83).

K-Fold Cross Validation

To ensure generalization, we used K-fold cross validation to test the model over multiple iterations and over multiple sets of data. Since K in this step is a hyperparameter, we iteratively tuned the variable until the average accuracy was maximized. This happened to occur when $K = 5$.

Model Training

In this step, we trained our models on the selected feature data vectors.

Model Optimization

Once trained, we would compare across different models and different hyperparameters to determine which model was optimal. Since we used K-fold cross validation, we did not have to worry very much about overfitting and could just worry about the accuracy percentage that the

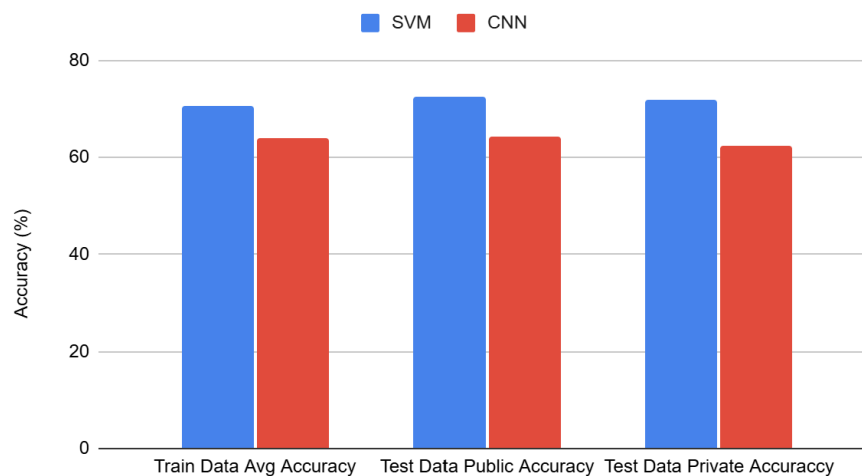
trained model produced. We would then iteratively optimize our models to obtain the highest possible accuracy percentage. Then, whichever model had the highest accuracy would be chosen.

Model Prediction

This model would then be evaluated on test data that it has never seen before. The results of this prediction were submitted to Kaggle to compare against other models from other teams.

6. Conclusion

SVM vs. CNN Model Performance



Based on the performance of our SVM and CNN models, we determined that our SVM model performed better overall on both training and test data. In our final kaggle submission, we choose to select the SVM model that was trained on a feature vector of length 100 and with the parameters: $C = 10$, kernel = rbf, and gamma = scale. We believed that these set of hyperparameters along with our feature select pipeline would have the optimal model accuracy while avoiding overfitting to the training data. When tested on the public dataset, our model accurately predicted emotions 72.535% of the time. Very similarly on the private dataset, it accurately predicted the correct emotion 71.853% of the time, proving that our model did not overfit to the train data. Given that the test data results matched the accuracy of our model with 5-fold cross validation (it actually had a higher accuracy than it), we can assume that the model generalized well due to the overfitting techniques that we implemented.

When looking at the features selected by rfe, they included mfccs, rmse, spectral contrast, zcr, spectral bandwidth, spectral flatness, and spectral rolloff, suggesting that these features played an important role in distinguishing emotions in audio files. Comparatively, features such as chroma, deltas, and spectral centroid were removed by rfe, suggesting that they were not as important. We believe that our model was able to outform some others due to the fact that we allowed the model to learn which features were important by itself instead of choosing them for it. Essentially, we gave up an abundant number of features and allowed it to recursively remove

features it deemed not important to end up with a set of features that accurately represented the variance in the data. If we had chosen arbitrary features, we would instead be relying on luck to find features that encompass the variation in the data. We believe that the feature selection process along with the recursive feature extraction played the largest role in improving the accuracy of our model. When we would alter the hyperparameters, we would often only see a few percentages in the change in accuracy; however, when adding new features that were important to the model or removing unnecessary ones with rfe, we would see larger percentage increases in the accuracy.

Overall, we were surprised by the importance of the role of feature extraction in accurately developing a model to classify data. Prior to this project, we had assumed that hyperparameter tuning and the specific model selected would play the largest role in determining how accurate our final model would turn out. In fact, we also incorrectly assumed that more complex models such as neural networks would outperform simpler models such as SVM or KNN. However, this project showed us that complex models do not necessarily outperform simpler models by large margins and in fact sometimes perform worse (due to overfitting), and that choosing optimal features would change accuracy to a greater degree than choosing an optimal machine learning model to represent the data. In the future and with more time, we could continue to optimize the feature extraction part of the data analysis pipeline. Specifically, we would attempt to reduce the dimensionality of our feature vector to speed up the model training process while keeping the majority of the variance captured by the data. This might require clever feature extraction or manipulation in order to create features without noise for our classifier model to better learn on and improve in accuracy. Finally, with more time, we would try to optimize our CNN model. Due to long running times, we were unable to optimize the parameters of our CNN model, which resulted in a lower accuracy than SVM. As a result of this, towards the end of the project, we decided to pivot to focus more on SVM. Given more time, we would choose to explore the CNN model more and test various optimization techniques to improve its accuracy to that of the SVM model.

7. Code Submission

<https://colab.research.google.com/drive/18wljMCV6ujUyLksxGkOpe8NDzMKJArVP?usp=sharing>

References

1. Feature Extraction - <https://www.mdpi.com/1424-8220/24/17/5797>, Section 3.3 - Feature Selection
2. MFCCs - <https://medium.com/@derutyosl/intuitive-understanding-of-mfccs-836d36a1f779>
3. Feature extraction with Librosa: <https://librosa.org/doc/0.10.1/feature.html>
4. Deltas - https://speechprocessingbook.aalto.fi/Representations/Deltas_and_Delta-deltas.html

5. Random Forest - https://en.wikipedia.org/wiki/Random_forest